Blockchain Fundamentals

Dr. Ahlad Kumar

NFSU Gandhinagar

December 2, 2025

Dr. Ahlad Kumar

Proof of Stake

- **Why PoS was Introduced:**
  - Proof of Work (PoW) ensures security but consumes massive energy and requires expensive hardware.
  - Miners compete using computational power, leading to centralization and environmental impact.
  - Proof of Stake (PoS) replaces computational effort with **economic stake**, achieving consensus efficiently.
- **Core Idea of PoS:**
  - Participants become **validators** by locking coins as collateral (stake).
  - Validators are chosen **randomly**, but the probability is proportional to their stake:

$$P(\text{Validator } i) = \frac{\text{Stake}_i}{\sum_{j=1}^{N} \text{Stake}_j}$$

  - Example: If Alice stakes 40, Bob 30, and Charlie 20 coins, Alice has a higher probability but is not guaranteed to be selected.
- **Key Intuition:**
  - Validators have "skin in the game" — misbehavior leads to loss of funds (**slashing**).

Dr. Ahlad Kumar

Proof of Stake: Algorithm

- **Objective:** Select a validator to propose the next block using stake weight and cryptographic randomness.

- **Core Components:**
  - S_i: Stake amount of validator i
  - R: Random seed (generated via Verifiable Random Function – VRF)
  - N: Total number of validators
  - P_i: Probability of validator i being selected

$$P_i = \frac{S_i}{\sum_{j=1}^{N} S_j}$$

- **Randomness Generation:**
  - The blockchain generates a random seed R at each round (or epoch).
  - Each validator computes a hash value:

$$T_i = H(V_i, R)$$

where H is a secure hash function.

Dr. Ahlad Kumar

Continued...

Algorithm: Validator Selection and Block Finalization

1. **Stake Phase:** Each validator V_i locks stake S_i into the staking contract.
2. **Selection Phase:** Using random seed R, determine proposer(s) via the rule $T_i < P_i$.
3. **Proposal Phase:** Selected validator creates block B containing pending transactions and signs it digitally.
4. **Broadcast Phase:** Validator sends B to all peers for verification.
5. **Attestation Phase:** Other validators verify B:
   - Check transaction validity, signatures, and previous hash. Vote (*attest*) if valid.
6. **Finalization Phase:**
   - When votes reach required quorum (e.g., $> 2/3$ majority), B is finalized. Once finalized, it cannot be reverted.
7. **Reward/Penalty Phase:**
   - Honest validators receive block rewards and transaction fees.
   - Malicious or offline validators face **slashing** (stake reduction).

What is V_i?

- V_i denotes a **unique identifier** for validator i. It is the identity element used when computing the per-validator random value.
- Typical choices for V_i:
  - the validator's **public key** (common and recommended),
  - the validator's **address** derived from the public key, or
  - a protocol-assigned **validator ID**.
- V_i is included in the hash so the random output is **bound to that specific validator**. This prevents a validator from reusing or claiming someone else's random output or selection.

Dr. Ahlad Kumar

Why hash V_i with the randomness R?

- The protocol generates a fresh random seed R each selection round (slot or epoch). This seed provides per-round unpredictability.
- Each validator computes a value from V_i and R so every validator obtains a different, unpredictable number tied both to their identity and the current round.
- The usual operation is:

$$H_i = H(V_i \,\|\, R)$$

  where H is a secure hash function (e.g., SHA-256) and $\|$ denotes concatenation or a canonical, domain-separated encoding of V_i and R.
- This produces a pseudorandom, validator-specific output that cannot be predicted or biased before R is known.

Dr. Abhad Kumar

Turning the hash into a selectable number

Goal: convert the hash output into a uniform number you can compare to the validator's stake fraction P_i.

Notation:

- $H_i = H(V_i \| R)$ — hash output (bit-string)
- X_i — integer interpretation of H_i (big-endian)
- K — hash bit-length (e.g., $K = 256$ for SHA-256)
- $P_i = \dfrac{S_i}{\sum_j S_j}$ — validator i's stake fraction

Equation Explained

1. Interpret H_i as an integer $X_i \in [0, 2^K - 1]$.
2. Compute the fractional value
$$T_i \;=\; \frac{X_i}{2^K} \quad \in [0, 1).$$
3. Select validator $i \iff T_i < P_i$.

This produces a uniformly distributed T_i on $[0, 1)$; the probability that T_i falls into $[0, P_i)$ equals P_i.

Dr. Ahlad Kumar

Example: Selection of validators, stakes and probabilities

- We have three validators who have staked tokens. Total stake = 100 coins.
- Each validator's **stake fraction** P_i is their stake divided by total stake.

Validator	Stake (coins)	Stake fraction P_i
Alice	50	$P_{\text{Alice}} = 0.50$
Bob	30	$P_{\text{Bob}} = 0.30$
Charlie	20	$P_{\text{Charlie}} = 0.20$

- Stake fraction = size of your "lottery window" each round.

Dr. Ahad Kumar

Continued...

- For this round the network publishes a fresh random seed:

$$R = \text{``ROUND_SEED_2025''}$$

- Each validator computes a hash of their identifier and the seed:

$$H_i = \mathrm{SHA256}(V_i \,\|\, R)$$

- In this classroom example we show only a **truncated portion** of each hash and map it to a small integer range for readability.

Validator	Input to hash	Truncated hash (hex)	Demo integer X_i	Normalized $T_i = X_i/65535$
Alice	`VA + ROUND_SEED_2025`	`3A4F ...`	14927	0.23
Bob	`VB + ROUND_SEED_2025`	`C1B8 ...`	49592	0.76
Charlie	`VC + ROUND_SEED_2025`	`7D22 ...`	32002	0.49

- **Note:** In real systems the hash is 256 bits and $T_i = X_i/2^{256}$. We use 16-bit scaling here (divide by 65 535) to keep numbers easy to compute and explain.

Dr. Ahlad Kumar

Continued...

- Selection test for each validator:

$$\text{select validator } i \quad \text{iff} \quad T_i < P_i$$

- Plugging in the demo values:

Validator	T_i (demo)	P_i (stake fraction)	Eligible?
Alice	0.23	0.50	✓ Eligible
Bob	0.76	0.30	× Not eligible
Charlie	0.49	0.20	× Not eligible

Dr. Ahlad Kumar

- **Result:** Alice is the only eligible validator this round, so she is chosen to propose the next block.
- The hash gives each validator a random ticket number $T_i \in [0, 1)$. If your ticket falls inside your ticket-window P_i, you are eligible. Bigger stake bigger window higher chance.

Reusability of Stake in Proof of Stake (PoS)

- In Proof of Stake, validators are required to **lock a certain number of coins** as collateral to participate in block validation.
- These coins are not destroyed but remain **temporarily locked** to ensure accountability and honest participation.
- Following the completion of a validation round or epoch, the staked amount may be **retained for continued use** or **released for withdrawal**, depending on the protocol design.
- The reusability or return of the staked amount is determined by the **staking smart contract** or the network's **consensus protocol rules**.

Dr. Ahad Kumar

Model Type	Behavior	Representative Systems
Reusable Stake	Remains locked and reused across epochs until the validator exits voluntarily.	Ethereum, Cardano, Polkadot
Time-Bound Stake	Locked for a fixed period; must be re-staked to continue participation after expiration.	BNB Chain, DeFi staking contracts

In PoS systems, the staked amount serves as **economic collateral** rather than an expendable resource. Its reusability or release is governed by the underlying **smart contract logic** or **protocol parameters**.

Dr. Ahlad Kumar

Proof of Burn

- Proof of Burn (PoB) lets participants earn the right to create blocks by **permanently destroying (burning)** some of their coins.
- A burn transaction is sent to an address with no private key — coins are visible on-chain but can never be recovered.
- The more coins a participant burns, the higher their chance of being chosen as the next block creator.
- This replaces the energy cost of PoW or the locked stake of PoS with an *economic sacrifice*.

Goal: Select a validator fairly based on burned value and randomness, then reward them for producing a valid block.

Dr. Ahlad Kumar

Proof of Burn

Algorithm: PoB Consensus – Setup and Eligibility

1. Each participant decides how many coins to burn and broadcasts a **burn transaction**.
2. Network verifies burns and records them permanently.
3. Compute total coins burned:
$$\text{TotalBurn} = \sum_i \text{BurnTx}[i]$$
4. For every participant i,
$$\text{BurnFraction}[i] = \frac{\text{BurnTx}[i]}{\text{TotalBurn}}$$
(acts as that participant's probability weight).
5. Network generates a fresh **random seed** R for this round.
6. Each participant combines their ID and the seed, then hashes the pair:
$$H_i = \text{Hash}(V_i \parallel R)$$
7. Convert hash to a random fraction in $[0, 1)$: $T_i = \text{Integer}(H_i)/2^K$

Proof of Burn: Continued...

Algorithm: PoB Consensus – Selection and Reward

8. If $T_i < \text{BurnFraction}[i]$, participant i is **eligible** to propose a block.
9. If several are eligible, choose the one with the smallest T_i; if none qualify, generate a new seed R and repeat.
10. The selected participant builds the new block:
    - Collects and validates transactions.
    - Broadcasts the proposed block.
11. Other validators verify the block's contents and signatures.
12. If accepted by the network:
    - Add block to blockchain.
    - Reward the winner with new coins or fees.
13. Burned coins remain destroyed forever — ensuring attacks are economically irrational.

Summary: Everyone burns coins → randomness picks the winner weighted by burn amount → winner adds block → earns reward → burned coins prove commitment.

Example

Goal: Show how Proof of Burn selects the block creator based on burned coins and random values.

- Each participant voluntarily burns (destroys) coins as proof of commitment.
- The higher the burn, the higher the chance of being selected.

Participants and Burns:

Participant	Coins Burned	Burn Fraction B_i
Alice	40	0.40
Bob	30	0.30
Charlie	30	0.30

Random Seed: $R =$ `"ROUND_SEED_POB"`

Dr. Allad Kumar

Random Ticket Values (Simulated Hash Fractions):

Participant	Input	Simulated Hash Output	Ticket T_i
Alice	`"VA + R"`	`3A4F...`	0.26
Bob	`"VB + R"`	`C1B8...`	0.48
Charlie	`"VC + R"`	`7D22...`	0.15

Rule: Participant is eligible if $T_i < B_i$

Dr. Ahlad Kumar

Continued

Eligibility Check:

Participant	T_i	B_i	Eligible?
Alice	0.26	0.40	✓ Yes
Bob	0.48	0.30	× No
Charlie	0.15	0.30	✓ Yes

Winner: Between eligible participants, the one with the smallest T_i wins.

$$T_{\text{Charlie}} = 0.15 < T_{\text{Alice}} = 0.26 \Rightarrow \textbf{Charlie is selected.}$$

Block Creation:

- Charlie builds and broadcasts the new block.
- Alice and Bob verify and attest the block.
- Once accepted, Charlie receives the block reward (e.g., 5 coins).
- Burned coins remain destroyed forever — proof of honesty.

Dr. Ahlad Kumar

Reward Mechanism in Proof of Burn

Concept: In PoB, only the selected validator earns a reward for producing a valid block. Burned coins are never returned, but long-term rewards can exceed the burn cost.

Reward Components:

$$\text{Total Reward} = \text{Block Reward} + \text{Transaction Fees}$$

Example (Continuing Our Case):

Participant	Coins Burned	Eligible?	Winner?	Reward (coins)
Alice	40	Yes	No	0
Bob	30	No	No	0
Charlie	30	Yes	Yes	5

Net Gain/Loss for Charlie:

$$\text{Net Gain} = \text{Reward} - \text{Burned Coins} = 5 - 30 = -25$$

(Short-term loss, but may profit over time with repeated wins.)

Dr. Ahlad Kumar

Long-Term View:

- Burn grants mining rights for multiple rounds.
- Frequent wins recover burn cost and generate profit.
- Typical block reward: fixed (e.g., 5 coins) + transaction fees.

Burning coins = buying a non-refundable lottery ticket. You lose the ticket price, but may earn rewards if you keep winning over time.

Dr. Arvind Kumar

Concept of Difficulty and Target

Definition: Difficulty measures how hard it is to find a valid block hash.

$$H(\text{block header}) \leq T$$

Relationship:

$$D = \frac{T_1}{T}$$

where:

- T_1 = fixed difficulty-1 target (genesis constant)
- T = current target

Difficulty-1 Target:

- Hard-coded in Satoshi's genesis block: `nBits = 0x1d00ffff`
- Decimal:

$$T_1 = 26959535291011309493156476344723991336010898738574164086137773096960$$

- Used as a reference for all difficulty calculations.

Intuition: Smaller target $T \Rightarrow$ harder to find valid hash $\Rightarrow$ higher difficulty.

Dr. Ahlad Kumar

Bitcoin Retargeting Rule

Bitcoin adjusts difficulty every 2016 blocks to maintain ≈10-minute block time.

1. **Retargeting in Target Space:**

2. **Retargeting in Difficulty Space:**

Dr. Ahlad Kumar

Bitcoin: Retargeting in Target Space

Goal: Adjust the mining *target* every 2016 blocks so that the average block time stays 10 minutes.

Target Update Formula:

$$T_{\text{new}} = T_{\text{old}} \times \frac{\text{ActualTime}}{\text{ExpectedTime}}$$

where

$$\text{ExpectedTime} = 2016 \times 10 \text{ minutes} = 20{,}160 \text{ minutes}.$$

Clamp Rule (Bounds):

$$\frac{1}{4} \leq \frac{\text{ActualTime}}{\text{ExpectedTime}} \leq 4$$

So in one adjustment the target can at most *quadruple* or be reduced to $\frac{1}{4}$ of its previous value.

Intuition:

- If miners collectively work **faster** than expected (ActualTime $<$ ExpectedTime), then $\frac{\text{ActualTime}}{\text{ExpectedTime}} < 1$ and $T_{\text{new}} < T_{\text{old}}$: the target **shrinks** (harder).
- If mining is **slower** (ActualTime $>$ ExpectedTime), then $T_{\text{new}} > T_{\text{old}}$: the target **expands** (easier).

Dr. Ahad Kumar

Short Numeric Example:

- Suppose $T_{\text{old}} = 166{,}666$ and ExpectedTime $= 20{,}160$.
- If blocks were mined in half the expected time: ActualTime $= 10{,}080$

$$T_{\text{new}} = 166{,}666 \times \frac{10{,}080}{20{,}160} = 83{,}333$$

(target halved $\rightarrow$ difficulty doubled).

- If blocks were $1.5\times$ slower: ActualTime $= 30{,}240$

$$T_{\text{new}} = 166{,}666 \times \frac{30{,}240}{20{,}160} = 250{,}000$$

(target increases $\rightarrow$ difficulty decreases).

Dr. Ahlad Kumar

Bitcoin: Retargeting in Difficulty Space

Concept: Bitcoin adjusts the mining difficulty every 2016 blocks to keep the average block time 10 minutes.

Recall Relationship:

$$D = \frac{T_1}{T}$$

where:

- T_1 = fixed difficulty-1 target (constant reference)
- T = current target value

Derivation:

$$\begin{aligned} D_{\text{new}} &= \frac{T_1}{T_{\text{new}}} \\ &= \frac{T_1}{T_{\text{old}} \times \frac{\text{ActualTime}}{\text{ExpectedTime}}} \\ &= D_{\text{old}} \times \frac{\text{ExpectedTime}}{\text{ActualTime}} \end{aligned}$$

Dr. Ahlad Kumar

Interpretation:

- If blocks are mined **too fast** (ActualTime $<$ ExpectedTime) $\Rightarrow D_{\text{new}} > D_{\text{old}} \rightarrow$ difficulty increases.
- If blocks are mined **too slow** (ActualTime $>$ ExpectedTime) $\Rightarrow D_{\text{new}} < D_{\text{old}} \rightarrow$ difficulty decreases.

Clamp Rule:

$$\frac{1}{4} \leq \frac{\text{ActualTime}}{\text{ExpectedTime}} \leq 4$$

Prevents difficulty from changing more than $4\times$ in one adjustment period.

Dr. Ahlad Kumar

(a) Choose initial difficulty and target

1. Choose an initial difficulty D_0 (common choice: $D_0 = 1$).
2. Compute the initial target from Satoshi's reference T_1:

$$T_{\text{initial}} = \frac{T_1}{D_0}.$$

3. Encode T_{initial} in the genesis header (compact form `nBits`) so miners can begin.
4. Start mining the first 2016 blocks using T_{initial}.

Dr. Ahlad Kumar

(b) Measure retarget after first 2016 blocks

Measure: After mining 2016 blocks, record the elapsed time ActualTime (minutes). Expected time is $\text{ExpectedTime} = 2016 \times 10 = 20{,}160$ minutes.

Compute new target:

$$T_{\text{new}} \;=\; T_{\text{initial}} \times \frac{\text{ActualTime}}{\text{ExpectedTime}} \;=\; T_{\text{initial}} \times \frac{\text{ActualTime}}{20{,}160}.$$

Apply clamp (safety bounds):

$$\frac{1}{4} \le \frac{\text{ActualTime}}{\text{ExpectedTime}} \le 4$$

If the raw ratio falls outside these bounds, clamp it to the nearest bound before computing T_{new}.

Dr. Ahlad Kumar

(c) Compute new difficulty and continue

1. Compute the new difficulty from the new target:

$$D_{\text{new}} = \frac{T_1}{T_{\text{new}}}.$$

2. Store T_{new} (compact `nBits`) for the next 2016-block interval.
3. Repeat the procedure every 2016 blocks:

$$T_{\text{next}} = T_{\text{new}} \times \frac{\text{ActualTime}_{\text{next}}}{20{,}160}, \qquad D_{\text{next}} = \frac{T_1}{T_{\text{next}}}.$$

At genesis you *must* set T_{initial} explicitly (e.g., T_1 for $D_0 = 1$). You cannot derive a target solely from elapsed time without having a prior target to scale.

Dr. Ahlad Kumar

Energy Consumption:Why Bitcoin Consumes Energy

- Bitcoin uses the **Proof of Work (PoW)** consensus mechanism.
- Miners compete to solve a cryptographic puzzle:

$$H(\text{block header}) \leq T$$

- To find a valid hash, miners must repeatedly perform **SHA-256 computations** by varying the nonce.
- Each attempt consumes electricity — only one miner wins per block, while all others' work (and energy) is wasted.

Energy cost ensures fairness and prevents spam — solving the puzzle requires measurable work.

Dr. Ahlad Kumar

What Energy Is Used For

Energy Consumption Components:

- **Computational Work:** Billions of SHA-256 hash calculations per second (*hashrate*).
- **Cooling Systems:** Mining rigs generate large amounts of heat; extra energy is required to cool them.
- **Maintenance & Infrastructure:** Includes data centers, ventilation, networking, and logistics.

Dr. Ahlad Kumar

3. Quantifying Bitcoin's Energy Usage

Formula:

$$E = P \times H$$

where:

- P = Power consumption per hash (Joules/hash)
- H = Global network hashrate (hashes/second)
- E = Energy consumed per second (Watts)

Example:

$$H = 500 \text{ EH/s} = 5 \times 10^{20} \text{ hashes/s}$$

$$P = 25 \text{ J/TH} = 2.5 \times 10^{-11} \text{ J/hash}$$

$$E = 5 \times 10^{20} \times 2.5 \times 10^{-11} = 1.25 \times 10^{10} \text{ J/s} = 12.5 \text{ GW}$$

That's roughly equivalent to the **power usage of a small country.**

Dr. Ahlad Kumar

4. Energy per Block and per Transaction

Given:

- Block mining time = 10 minutes = 600 seconds
- Network energy per second 12.5 GW

Energy per block:

$$E_{\text{block}} = 12.5 \times 10^9 \times 600 = 7.5 \times 10^{12} \text{ J}$$

7.5 terajoules (TJ) per block.

If each block includes 2,000 transactions:

$$E_{\text{tx}} = \frac{7.5 \times 10^{12}}{2000} = 3.75 \times 10^9 \text{ J}$$

$\approx$ **1,000 kWh per transaction.**

Energy use varies with network hashrate and mining hardware efficiency.

Dr. Ahlad Kumar

Why and Improvements

Why So Much Energy?

- **Security:** High energy cost deters attackers — to rewrite the chain, they would need to control >51% of total hashrate.
- Energy acts as an **economic barrier** — cheating becomes prohibitively expensive.
- Energy expenditure guarantees network honesty and immutability.

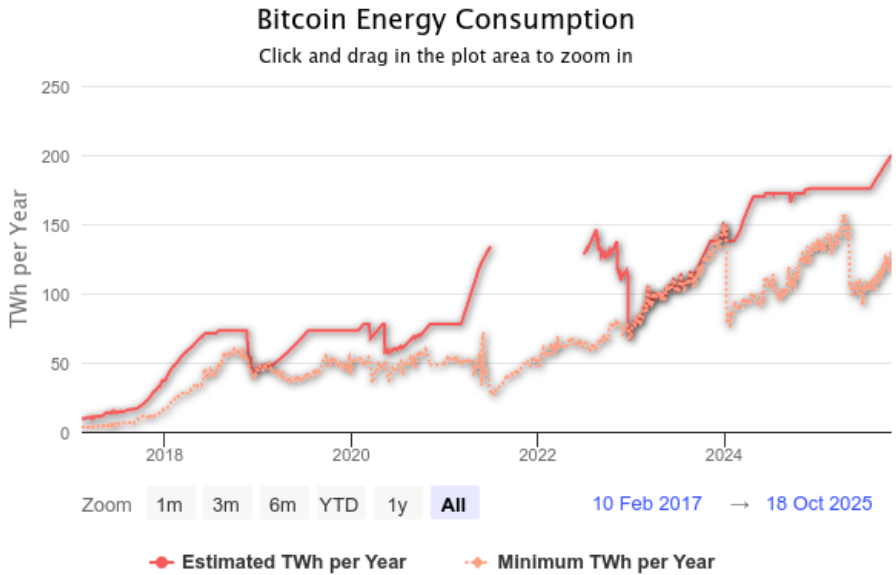
Efficiency Improvements:

- New ASICs improve efficiency (lower J/TH, more hashes per watt).
- Many mining farms now use **renewable energy sources** (hydro, wind, solar).
- Future models (e.g., **Proof of Stake**) aim to retain security with much lower energy use.

Conclusion: Bitcoin's energy use is high, but it underpins trust through **economic and physical cost.**

Dr. Ahlad Kumar

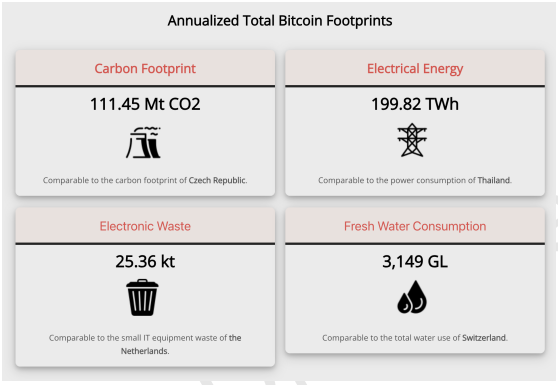
Bitcoin Energy Consumption (Estimated vs Minimum)

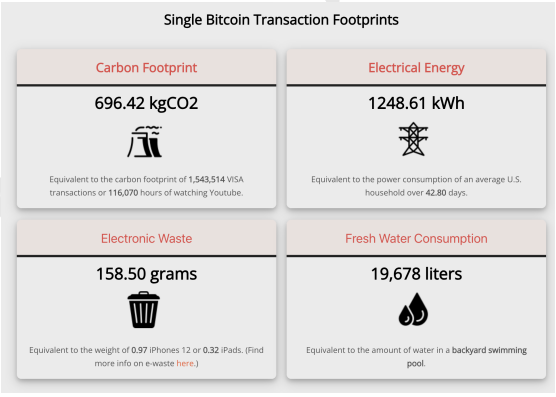


- **Axes and units:** vertical axis = energy consumption (TWh per year); horizontal axis = time (2017 $\rightarrow$ 2025).
- **Red (Estimated TWh/yr):** realistic estimate based on the network hashrate and a weighted average of mining hardware efficiencies — a best-guess of actual electricity use.
- **Orange dotted (Minimum TWh/yr):** theoretical lower bound assuming every miner runs the *most efficient* current ASICs; represents the best-case energy use for the observed hashrate.

Dr. Amad Kumar

Bitcoin Energy Comparison (Side-by-Side)





Nakamoto Consensus : (a) Mining and Proof of Work

- Every node (**miner**) collects valid transactions into a candidate block.
- Each miner searches for a hash of the block header that satisfies the Proof of Work condition:

$$H(\text{block header}) \leq T$$

  where T is the **difficulty target**.
- The first miner to find such a valid hash broadcasts its block to the network.

Concept:

- This process requires enormous computational effort.
- The probability of finding a valid block is proportional to a miner's computational power.

Dr. Ahlad Kumar

Nakamoto Consensus : (b) Validation and Propagation

- Other nodes in the network verify the received block.
- Verification includes:
  - Checking that all transactions are valid and properly signed.
  - Ensuring the Proof of Work is correct ($H(\text{block header}) \leq T$).
  - Confirming that the previous block hash links correctly to the existing blockchain.
- If all checks pass, nodes append the block to their local copy of the blockchain.

Purpose: This decentralized validation ensures that only legitimate blocks are accepted and propagated throughout the network.

Dr Ahlad Kumar

Nakamoto Consensus : (c) Longest Chain Rule

- Occasionally, two miners find valid blocks almost simultaneously — creating a **temporary fork**.
- Nodes may temporarily disagree on which block is the most recent.
- As new blocks are added, one branch accumulates more cumulative Proof of Work.
- Nodes always follow the **Longest (Heaviest) Chain Rule**:
  - The chain with the greatest cumulative PoW is considered the valid blockchain.
  - All nodes switch to this longest chain, ensuring network-wide convergence.

Result: Eventually, the network reaches consensus even if short-lived forks occur.

Dr. Ahlad Kumar

Nakamoto Consensus : (d) Incentives for Honesty

- Honest miners receive:
  - **Block rewards** (newly minted coins), and
  - **Transaction fees** from the transactions in the block.
- Dishonest miners waste:
  - Energy and computational resources on invalid blocks.
  - Their work is rejected by the network and yields no reward.
- Therefore, being honest is the most **economically rational strategy**.
- Incentives align miners' self-interest with the health and security of the network.

Summary: Nakamoto Consensus achieves trust through economic incentives and computational effort rather than central authority.

Dr. Ahlad Kumar